Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress · Checkpoint 5

Lívia

TU Munich · Siemens

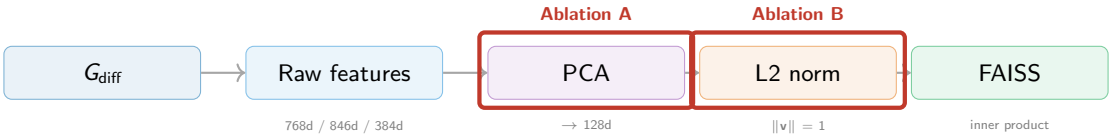
May 12, 2026

Agenda

- **Checkpoint 4 recap** — PCA & L2 ablation results
- **Embedding fusion strategies [Updated results]** — 3-way combination & 4-way CodeBERT experiment
- **Embedding space analysis updated results** — single-space & cross-space diagnostics
- **Agent patching: First Variations and Insights** — LLM patch generation by analogy
- **Prompt variation experiment** — 4 prompt variants on 41-query subset
- **Key findings & next steps**

Checkpoint 4 Recap

Pipeline validation: PCA & L2 normalization



Combination Strategies: Experiment Design

Goal

The Combined embedder concatenates NetLSD + WL + GIN then reduces with PCA. Does pre-processing each sub-embedder *before* concatenation improve the fused space?

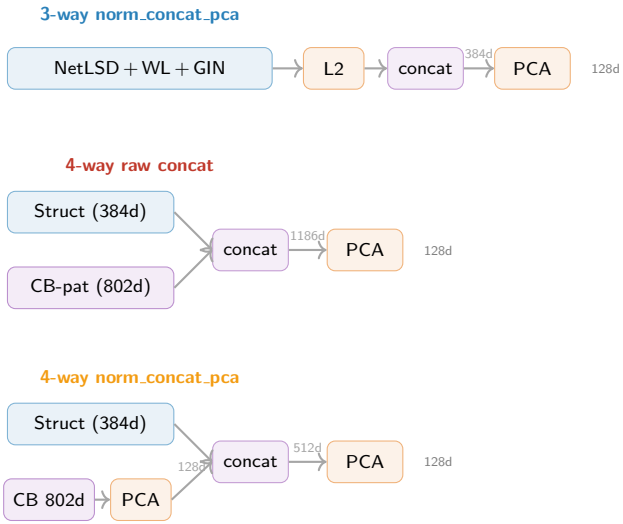
Strategies compared

Strategy	Pipeline
`concat_pca`	Concatenate raw → PCA to 128d (baseline)
`pca_concat_pca`	PCA each to 128d → concat (384d) → PCA to 128d
`pca_concat`	PCA each to 42d → concat to 126d (no 2nd PCA)
`norm_concat_pca`	L2-norm each → concat → PCA to 128d

Combination Strategies: Updated Results (n=73)

	Retrieval					
Strategy	hit@1	hit@5	hit@10	MRR	nDCG@10	MAP@10
`norm_concat_pca`	80.8%	93.2%	95.9%	.849	.645	.514
`concat_pca`	64.4%	79.5%	84.9%	.707	.452	.336
`pca_concat_pca`	64.4%	79.5%	84.9%	.707	.452	.336
`pca_concat`	64.4%	78.1%	83.6%	.706	.449	.334
4-way fusion (+ CodeBERT-pattern)						
`4way_concat_pca`	64.4%	79.5%	84.9%	.707	.452	.336
`4way_norm_concat_pca`	72.6%	86.3%	87.7%	.773	.569	.440

4-Way Fusion: Pipeline Comparison



Visualizing the Embedding Space: Methodology

Goal

Understand how different embedding strategies structure the vulnerability space and whether PCA preserves neighbourhood quality.

Embedders compared

- **CodeBERT-pattern** — CodeBERT applied to extracted code *patterns* from the diff graph (802d raw: 34d pattern + 768d CodeBERT)
- **Combined** — NetLSD + WL + GIN structural embedders concatenated (384d raw)

Visualization pipeline

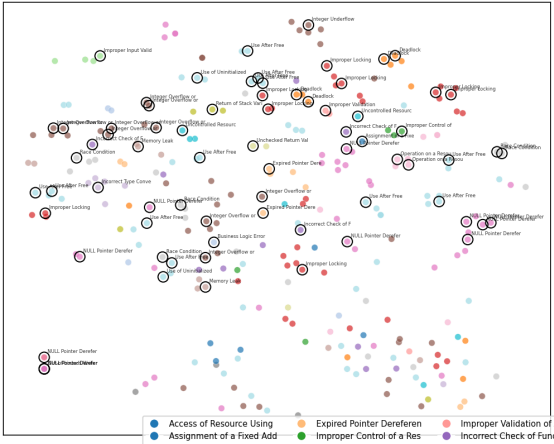
1. Extract embeddings for all CVE variants
2. For each embedder, produce two views:
   - **Raw** (high-dim, L2-normed) → t-SNE to 2D
   - **PCA→128d + L2** → t-SNE to 2D
3. Color points by **CWE class**; mark query samples with black open circles

ℹ Side-by-side comparison reveals whether PCA dimensionality reduction preserves or improves CWE cluster structure.

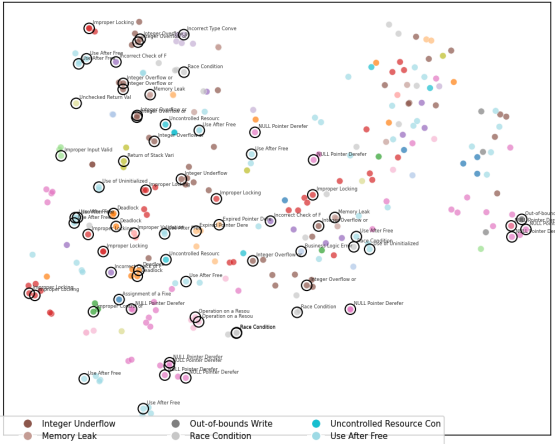
Space Analysis

Embedding Space: codebert_pattern (n=298, seed=42)

codebert_pattern — Raw (802d, L2-normed)



codebert_pattern — PCA→128d + L2

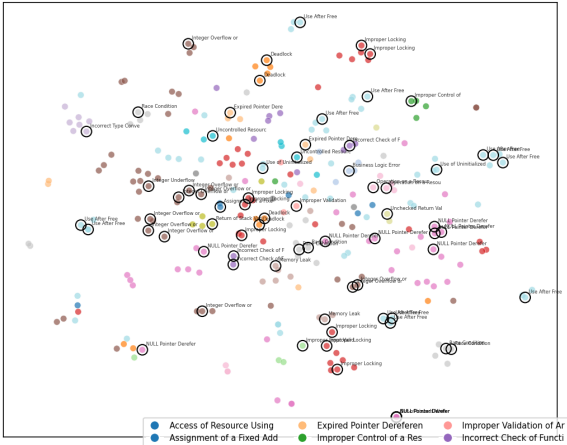


Access of Resource Using
Assignment of a Fixed Add
Business Logic Errors
Deadlock
Expired Pointer Dereferen
Improper Control of a Res
Improper Input Validation
Improper Locking
Improper Validation of Ar
Incorrect Check of Functi
Incorrect Type Conversion
Integer Overflow or Wrapa
Integer Underflow
Memory Leak
NULL Pointer Dereference
Operation on a Resource a
Out-of-bounds Write
Race Condition
Return of Stack Variable
Unchecked Return Value
Uncontrolled Resource Con
Use After Free
Use of Uninitialized Vari
query sample

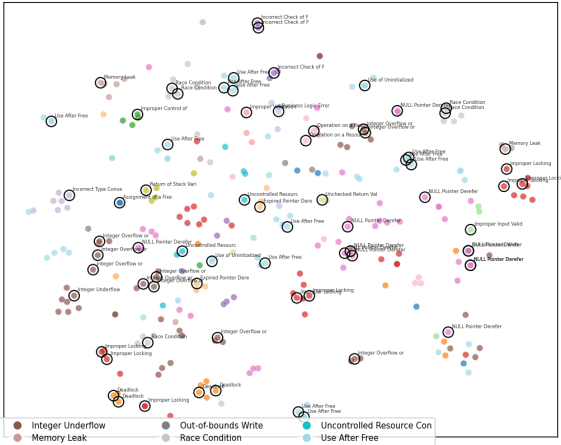
Space Analysis

Embedding Space: combined (n=298, seed=42)

combined — Raw (384d, L2-normed)



combined — PCA→128d + L2



Space Analysis: Interpretation

CodeBERT-pattern (802d → 128d)

- Distinct clusters for NULL Pointer Dereference and Uncontrolled Resource Consumption — CodeBERT captures code-level vulnerability semantics
- Central region is "muddy": Business Logic Errors and Improper Input Validation overlap, indicating these CWEs share similar code patterns
- Query samples (black circles) land on top of same-class index samples — good k-NN retrieval despite cluttered global structure

Combined vs. CodeBERT-pattern

- CodeBERT-pattern produces tighter CWE clusters; Combined (structural) is more blended — code-semantic features separate vulnerability types more clearly than graph-structural features
- PCA (802d → 128d) preserves cluster structure in CodeBERT-pattern with 99.4% variance explained, confirming the raw space is largely low-rank
- Despite better visual clustering, CodeBERT-pattern retrieval (hit@1=71.2%) trails Combined (hit@1=89.0%) — local neighbourhood quality matters more than global separation

GIN Training: Experiment Design

Goal

Can we improve the frozen GIN embedder (random weights, seed-42) by fine-tuning with **triplet metric learning**? Compare two feature strategies: structural vs. CodeBERT. **Shared architecture**

`input_proj` $\rightarrow$ 3$\times$`GINConv(MLP) + BatchNorm`
$\rightarrow$ `add_pool` $\|$ `mean_pool` $\rightarrow$ `readout` $\rightarrow$ `L2`

Output: 128d L2-normalized.
Loss: `TripletMarginLoss(p=2)`, semi-hard mining every 5 epochs.

Optimizer: Adam, weight decay 10^{-4}, gradient clip 1.0.

Two variants

	GIN-Struct	**GIN-CodeBERT**
Node feat.	11d one-hot (AST type)	768d CodeBERT per node
Init	**Warm start** from frozen GIN weights	Random init
Triplet labels	CVE-ID (64 classes)	CWE-ID (23 classes)
Margin	0.5	0.3
LR / sched.	5×10^{-4} / cosine	10^{-3} / constant
Dropout	0.2	0.3

Warm start: copy frozen GIN's exact parameters, then fine-tune — preserves known-good structural geometry.

GIN Training: Results

Model	Epochs	Train loss	hit@1	hit@10	MRR	CWE recall	CVE F1
Frozen GIN (baseline)	—	—	72.6%	90.4%	.788	.443	—
GIN-Struct (trained)	94	.012	78.1%	94.5%	.826	.578	.808
GIN-CodeBERT (trained)	42	.151	28.8%	57.5%	.372	.212	.296
Reference (untrained combined embedders)							
Combined (N+WL+GIN)	—	—	89.0%	97.3%	.916	.498	—

GIN-Struct: strong

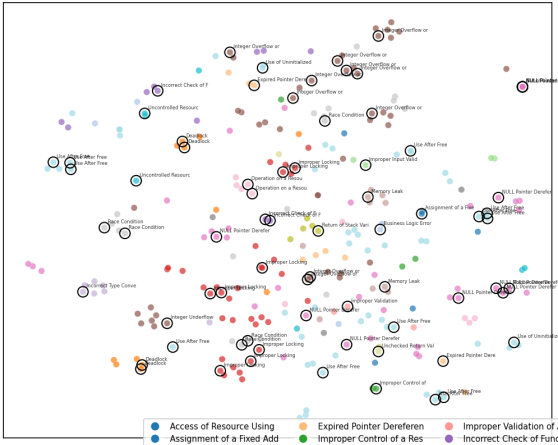
- Warm start + CVE-level triplets → hit@1 improves 72.6% → 78.1% (+5.5pp)
- CWE recall jumps 44.3% → 57.8% — better cross-class generalization
- Approaches Combined (89%) using a *single* embedder with only 11d node features

GIN-CodeBERT: failed

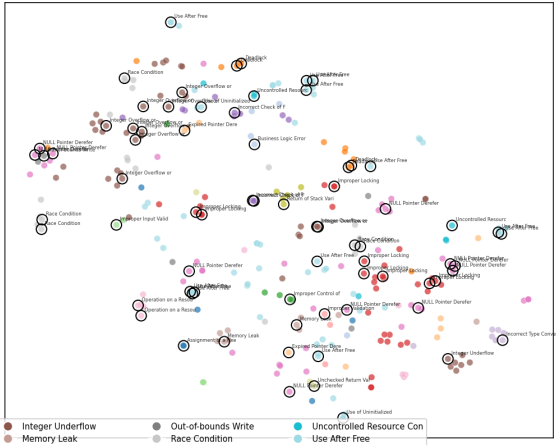
- 768d CodeBERT node features did not help — hit@1 drops to 28.8%, worse than frozen GIN
- Loss plateaus at 0.151 (vs. 0.012 for Struct), model does not converge
- CWE-level labels too coarse for CVE-level retrieval; random init loses structural geometry

GIN-Struct Raw vs PCA (n=298)

GIN-Struct — Raw (128d, L2-normed)



GIN-Struct — PCA→64d + L2



Access of Resource Using
Assignment of a Fixed Add
Business Logic Errors
Deadlock
Expired Pointer Dereferen
Improper Control of a Res
Improper Input Validation
Improper Locking
Improper Validation of Ar
Incorrect Check of Functi
Incorrect Type Conversion
Integer Overflow or Wrapa
Integer Underflow
Memory Leak
NULL Pointer Dereference
Operation on a Resource a
Out-of-bounds Write
Race Condition
Return of Stack Variable
Unchecked Return Value
Uncontrolled Resource Con
Use After Free
Use of Uninitialized Vari
query sample

[RECAP] Agent Baseline: Experiment Design

Goal

Given a vulnerable function and a retrieved similar CVE, can an LLM generate a correct patch **by analogy**? Compare *oracle* retrieval (perfect) vs. *embedding* retrieval (our pipeline).

Model & prompt

- **GPT-4o** (Azure), temperature=0.2, max_tokens=4096
- **One-shot patching by analogy**: system prompt contains the retrieved CVE's fix-list, variable/function mapping, root cause
- Assistant message shows the example's CoT + patched code
- User message provides the target function to patch
- Agent must output `[CoT START]...[Patched Code END]`

[RECAP] Agent Baseline: Per-CWE Oracle vs. Embedding (BERTScore & ROUGE-L)

CWE class	n	hit@1 (%)	Orac. BERT	Emb. BERT	Δ%	Orac. RG-L	Emb. RG-L	Δ%
Fixed Address to Pointer	1	100	.969	.969	0.0	.993	.993	0.0
Resource Lifetime	1	100	.934	.934	0.0	.951	.951	0.0
Memory Leak	2	100	.911	.902	−0.9	.926	.921	−0.6
Check Return Value	3	33	.904	.603	−33.3	.934	.613	−34.4
Use After Free	12	75	.861	.721	−16.2	.874	.748	−14.4
Uninitialized Variable	2	100	.841	.853	+1.4	.953	.961	+0.8
Return of Stack Var	1	100	.815	.820	+0.6	.883	.864	−2.1
Type Conversion	1	100	.802	.692	−13.6	.978	.978	0.0
Deadlock	4	100	.767	.726	−5.4	.833	.842	+1.1
Resource Expiration	2	50	.765	.725	−5.3	.854	.838	−1.8
Array Index Validation	1	0	.720	.882	+22.6	.745	.959	+28.7
Input Validation	1	0	.715	.715	0.0	.689	.689	0.0
Race Condition	5	100	.703	.712	+1.2	.689	.731	+6.1
Integer Overflow	10	60	.699	.705	+0.9	.829	.804	−3.1
Business Logic	1	100	.689	.750	+8.9	.702	.769	+9.6
Integer Underflow	1	100	.681	.726	+6.6	.840	.880	+4.7
NULL Pointer Deref	11	36	.629	.683	+8.6	.710	.723	+1.8
Expired Pointer Deref	2	100	.587	.583	−0.8	.466	.468	+0.5
Improper Locking	8	100	.565	.550	−2.6	.544	.525	−3.4
Resource Exhaustion	2	0	.539	.710	+31.7	.649	.665	+2.4
Unchecked Return Value	1	0	.428	.495	+15.6	.466	.603	+29.4
Out-of-bounds Write	1	0	.151	.008	−94.6	.454	.670	+47.6
Overall	**73**	**68.5**	**.716**	**.691**	**−3.5**	**.766**	**.741**	**−3.3**

hit@1 = % queries where FAISS top-1 matches the correct CVE. 12/22 CWEs: embedding ≥ oracle Low hit@1 does **not** always

[RECAP] LLM Semantic Evaluation: Experiment Design

Goal

Text-similarity metrics (BLEU, Jaccard) measure *surface overlap* with the ground truth, not whether the vulnerability is actually fixed. Can an LLM judge the **semantic correctness** of a patch?

Rationale

A patch can be textually different yet semantically correct (e.g. different variable names, reordered guards). Conversely, a high-BLEU patch may miss the critical fix. We need a **vulnerability-aware** evaluation.

Scope

All 73 oracle-mode patches from the agent baseline, evaluated with GPT-4o as a security auditor.

Prompt design

1. **Role:** "Expert security code auditor specializing in vulnerability analysis"
2. **Input:** CVE ID, CWE class, vulnerability description, original vulnerable code, generated patch
3. **Task:** Determine if the patch fixes the specific vulnerability described
4. **Output format:** structured JSON with `verdict` (FIXED / NOT_FIXED / PARTIAL), `confidence` (0.0–1.0), `reasoning` (free text)

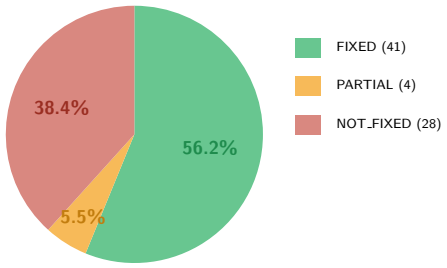
Model & parameters

- **GPT-4o** (Azure), temperature=0
- Deterministic evaluation (no sampling variance)
- 1.5s delay between calls (rate limiting)

[RECAP] LLM Semantic Evaluation: Preliminary Results (n=73)

Verdict	Count	%
FIXED	41	56.2%
PARTIAL	4	5.5%
NOT_FIXED	28	38.4%
Fix rate (FIXED + PARTIAL)	**45**	**61.6%**

Verdict	Avg. confidence
FIXED	0.948
NOT_FIXED	0.902
PARTIAL	0.750
Overall	**0.919**



Model: GPT-4o (Azure), temperature=0.

Run: `python -m src.evaluate.llm_evaluation results.jsonl`

[RECAP] LLM Semantic Evaluation: Per-CWE Breakdown

CWE class	n	hit@1 CVE (%)	FIXED	PARTIAL	NOT_FIXED	Fix rate
Array Index Validation	1	0	1	0	0	100%
Business Logic	1	100	1	0	0	100%
Input Validation	1	0	1	0	0	100%
Integer Underflow	1	100	1	0	0	100%
Out-of-bounds Write	1	0	1	0	0	100%
Resource Expiration	2	50	2	0	0	100%
Unchecked Return Value	1	0	1	0	0	100%
Uninitialized Variable	2	100	2	0	0	100%
Integer Overflow	10	60	7	2	1	90.0%
Use After Free	12	75	10	0	2	83.3%
Race Condition	5	100	3	0	2	60.0%
Deadlock	4	100	2	0	2	50.0%
Improper Locking	8	100	3	1	4	50.0%
Memory Leak	2	100	1	0	1	50.0%
Resource Exhaustion	2	0	1	0	1	50.0%
NULL Pointer Deref	11	36	3	1	7	36.4%
Check Return Value	3	33	1	0	2	33.3%
Expired Pointer Deref	2	100	0	0	2	0%
Fixed Address to Pointer	1	100	0	0	1	0%
Resource Lifetime	1	100	0	0	1	0%
Return of Stack Var	1	100	0	0	1	0%
Type Conversion	1	100	0	0	1	0%
Overall	**73**	**68.5**	**41**	**4**	**28**	**61.6%**

Prompt Variation: Experiment Design

Goal

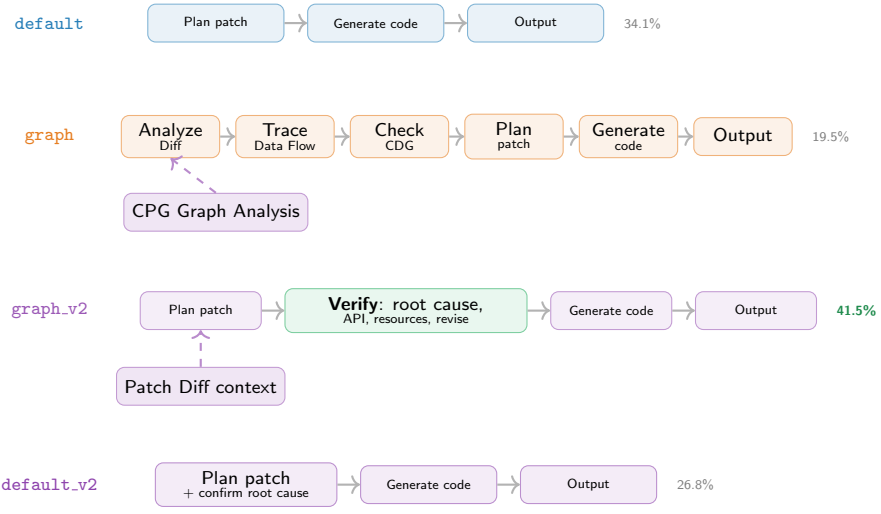
Measure how the agent's system prompt affects patch quality. Four variants alter CoT depth and whether graph context is injected into the target slot.

Setup

- Same 41-query subset (39 unique CVE/variant pairs)
- Oracle retrieval, GPT-4o, temperature=0.2
- Metrics: BLEU-4, BERTScore F1, ROUGE-L, Jaccard, LLM fix rate

Variant	Key difference
`default`	3-step CoT (describe → generate → output)
`graph`	6-step CoT + CPG analysis + graph context in target
`graph_v2`	Verification step + patch diff as graph context
`default_v2`	3-step CoT + explicit root-cause confirmation

Prompt Variation: Pipeline Comparison



Dashed arrows = graph context injection Green box = explicit verification step Percentages = LLM fix rate

Prompt Variation: Key Differences Summary

Aspect	default	graph	graph_v2	default_v2
CoT steps	3	6	4	3
CPG data in prompt	✗ None	✓ Full	− Diff only	✗ None
Verification checklist	✗	✗	✓	✗
Root-cause confirmation	✗	✗	✓ (in verify)	✓ (1 sentence)
Assistant shows verification	✗	✗	✓	✗
LLM fix rate	34.1%	19.5%	41.5%	26.8%

Full CPG = Patch Diff + Data Flow + Control Dependencies **Diff only** = Patch Diff section (lines changed, AST types)

Prompt Variation: Results (n=41)

Variant	BLEU-4	BERTScore F1	ROUGE-L	Tok. Jaccard	LLM Fix Rate
`default`	.652	.718	.752	.724	34.1% (14/41)
`graph`	.638	.728	.746	.718	19.5% (8/41)
`graph_v2`	.654	.723	.753	.722	41.5% (17/41)
`default_v2`	.638	.716	.740	.716	26.8% (11/41)

Text metrics

- All variants within 2.5% of each other
- `graph_v2` and `default` lead

LLM fix rate

- `graph_v2` leads at 41.5%
- `graph` drops to 19.5%

Divergence

- Text similarity $\neq$ semantic fix
- Verification step matters most

Prompt Variation: Analysis

- **Verification step is the strongest lever.** `graph_v2` adds "VERIFY root cause, API compliance, resource ownership" and reaches 41.5% fix rate — 2× the base `graph` variant (19.5%).
- **Raw graph context hurts without structure.** `graph` injects CPG analysis but achieves the lowest fix rate (19.5%), suggesting unstructured graph data distracts the model from the fix pattern.
- **Text metrics are near-indistinguishable.** ΔBLEU-4 across all variants is only 0.016 (2.5%). LLM-as-judge reveals quality differences invisible to surface-level n-gram overlap.
- **Root-cause confirmation alone is insufficient.** `default_v2` adds "confirm your plan addresses root cause" but only reaches 26.8% — the verification checklist in `graph_v2` is more actionable.

Key Findings

Retrieval hit@1: 89.0% overall, but varies by CWE. 12/22 CWEs have perfect retrieval; 8/22 have hit@1 $\leq$ 75%. Some CWEs (e.g. Resource Exhaustion) have low hit@1 but high BERT/RG-L, suggesting the embedding retrieves semantically relevant analogies even if not the exact CVE.

Embedding retrieval nearly matches oracle for patching. Oracle BLEU .655 / Embedding .637 (-2.7%); ROUGE-L .766 / .741 (-3.3%). Stereotyped CWEs (Use After Free, Integer Overflow) patch well; complex locking refactors remain hard (.34).

LLM semantic evaluation: 61.6% of patches fix the CVE. GPT-4o as security auditor confirms 41/73 fully fixed + 4 partial. Retrieval$\rightarrow$generation gap (89%$\rightarrow$62%) shows the bottleneck is fix transfer, not analogy retrieval.

Verification prompts double fix rate vs. raw graph context. `graph_v2` (41.5%) vs. `graph` (19.5%) on the same 41 queries (the hardest). Text metrics are near-identical (ΔBLEU 2.5%); LLM-as-judge exposes the real quality gap.

Next Steps

- **Analyse retrieval failure modes.** Embedding loses 14–16% on Use After Free — investigate which CVE variants are mis-retrieved and whether reranking helps.

Repository: `graph-rag/` `python -m experiments.agent_experiment`